Reordering Columns with dplyr + df

https://gemini.google.com/app/aa2d416e3c3f3830

User prompt: no can you add all of those elements in red but only for points with n>10?

Response: To highlight only the well-sampled years ($n > 10$), we can filter the data directly inside the new layers.

By adding a filtered `geom_line()` and `geom_point()` in red, they will overlay perfectly on top of your existing base layers—leaving your weighted GAM curve to utilize all the data.

Here is the updated code:

```
p2.L <- ggplot(data = mayW, aes(x = Year, y = Lmu)) +
  ggtitle("Length ~ HW + May/Jun avg") +

  # --- BASE LAYERS (All Data) ---
  # Background connecting line for all years
  geom_line(color = "grey70", alpha = 0.5) +
  # All points sized by 'n'
  geom_point(aes(size = n), color = "black", alpha = 0.4) +

  # --- HIGH SAMPLE SIZE HIGHLIGHTS (n > 10 only) ---
  # Red connecting line for years with n > 10
  geom_line(data = filter(mayW, n > 10), color = "firebrick", linewidth = 0.8) +
  # Red overlay points for years with n > 10
  geom_point(data = filter(mayW, n > 10), aes(size = n), color = "firebrick", alpha = 0.9) +

  # --- SMOOTHING LAYER (Weighted by 'n') ---
  geom_smooth(
    method = "gam",
    formula = y ~ s(x, bs = "cs"),
    aes(weight = n),
    color = "darkblue"
  ) +
  labs(size = "Sample Size (n)")

p2.L
```

Why this works so well:

- **`data = filter(mayW, n > 10)`**: Passing a filtered subset directly to specific geoms allows them to selectively draw *only* those rows.
- **Continuous Line Behavior**: `geom_line()` will cleanly bridge the gap across any omitted years where $n \leq 10$, keeping your high-sample timeline connected.
- **Clear Contrast**: Setting the base layers to a lighter opacity (`alpha = 0.4`) ensures your red high-observation data points immediately pop off the screen.

User prompt: if I can't see a red gam line, is that because it is right on top of the blue line from before, or is it not being plotted?

Response: It isn't being plotted! In the new code block I provided, **there is only one GAM line**, and I changed its color from the default light blue to `"darkblue"`.

Because `geom_smooth()` is still reading your entire dataset (`data = mayW` from the main `ggplot()` call), it is drawing a single, overall trendline that is weighted by your sample sizes.

If you actually wanted **two distinct GAM lines**—a blue one for the whole dataset and a red one built *exclusively* from your high-sample data ($n > 10$)—you have to tell `geom_smooth()` to filter the data just like we did for the points.

Here is how you plot both so you can visually compare how much those low-n years are pulling your curve:

```
p2.L <- ggplot(data = mayW, aes(x = Year, y = Lmu)) +
  ggtitle("Length ~ HW + May/Jun avg") +

  # --- ALL DATA (Blue Theme) ---
  geom_line(color = "grey70", alpha = 0.5) +
  geom_point(aes(size = n), color = "black", alpha = 0.4) +
  geom_smooth(
    method = "gam", formula = y ~ s(x, bs = "cs"), aes(weight = n),
    color = "darkblue", fill = "lightblue", alpha = 0.2
  ) +

  # --- HIGH SAMPLE DATA ONLY (Red Theme) ---
  geom_line(data = filter(mayW, n > 10), color = "firebrick", linewidth = 0.8) +
  geom_point(data = filter(mayW, n > 10), aes(size = n), color = "firebrick", alpha = 0.9) +
  geom_smooth(
    data = filter(mayW, n > 10), # <-- Drops low-n years entirely from this model
    method = "gam", formula = y ~ s(x, bs = "cs"),
    color = "firebrick", fill = "firebrick", alpha = 0.1
  ) +

  labs(size = "Sample Size (n)")

p2.L
```

What to look for now:

Now you will see both a dark blue curve and a red curve.

- If the red curve sits **exactly on top** of the blue curve, it means your low-sample years were already carrying so little weight in the blue model that dropping them entirely changed absolutely nothing.
- If the red curve diverges, you'll be able to see exactly how much those low-observation years were skewing your overall ecological trend!

User prompt: ok, now put them on the same page so I can see both plots side by side: mayW<-mu.length %>% filter(Month=="May",Tag=="NO") junH<-mu.length %>% filter(Month=="June",Tag=="YES") mayW.plot <- ggplot(data = mayW, aes(x = Year, y = Lmu)) + ggtitle("Length ~ HW + May/Jun avg") + # --- ALL DATA (Blue Theme) --- geom_line(color = "grey70", alpha = 0.5) + geom_point(aes(size = n), color = "black", alpha = 0.4) + geom_smooth(method = "gam", formula = y ~ s(x, bs = "cs"), aes(weight = n), color = "darkblue", fill = "lightblue", alpha = 0.2) + # --- HIGH SAMPLE DATA ONLY (Red Theme) --- geom_line(data = filter(mayW, n > 10), color = "firebrick", linewidth = 0.8) + geom_point(data = filter(mayW, n > 10), aes(size = n), color = "firebrick", alpha = 0.9) + geom_smooth(data = filter(mayW, n > 10), # <-- Drops low-n years entirely from this model method = "gam", formula = y ~ s(x, bs = "cs"), color = "firebrick", fill = "firebrick", alpha = 0.1) + labs(size = "Sample Size (n)") mayW.plot junH.plot <- ggplot(data = junH, aes(x = Year, y = Lmu)) + ggtitle("Length ~ HW + May/Jun avg") + # --- ALL DATA (Blue Theme) --- geom_line(color = "grey70", alpha = 0.5) + geom_point(aes(size = n), color = "black", alpha = 0.4) + geom_smooth(method = "gam", formula = y ~ s(x, bs = "cs"), aes(weight = n), color = "darkblue", fill = "lightblue", alpha = 0.2) + # --- HIGH SAMPLE DATA ONLY (Red Theme) --- geom_line(data = filter(mayW, n > 10), color = "firebrick", linewidth = 0.8) + geom_point(data = filter(mayW, n > 10), aes(size = n), color = "firebrick", alpha = 0.9) + geom_smooth(data = filter(mayW, n > 10), # <-- Drops low-n years entirely from this model method = "gam", formula = y ~ s(x, bs = "cs"), color = "firebrick", fill = "firebrick", alpha = 0.1) + labs(size = "Sample Size (n)") junH.plot

Response: To stitch these two plots together side-by-side cleanly, the best tool in R is the **`patchwork`** package. It is incredibly lightweight and lets you arrange plots using simple math symbols like `+` or `/`.

First, make sure you have it installed (`install.packages("patchwork")`), and then you can add this block right below your code to display them on the same page:

```
library(patchwork)

# 1. Update the title on the second plot so you can easily tell them apart!
mayW.plot <- mayW.plot + ggtitle("May (Tag: NO) - Length ~ HW")
junH.plot <- junH.plot + ggtitle("June (Tag: YES) - Length ~ HW")

# 2. Put them side-by-side and collect the legends so it looks clean
combined_plots <- (mayW.plot + junH.plot) +
  plot_layout(guides = "collect")

# 3. Print the final layout
combined_plots
```

Why this approach works beautifully:

- **`+` operator:** `patchwork` intercepts the plus sign when you combine two ggplot objects, naturally placing them side-by-side.
- **`guides = "collect"`:** Since both of your plots use the exact same `Sample Size (n)` scale, this automatically merges them into a single legend on the right margin instead of cluttering your screen with two identical legends.

(Alternative tip: If you don't want to install a new package, you can use the base R alternative `cowplot::plot_grid(mayW.plot, junH.plot, ncol = 2)`, but patchwork handles the legends much cleaner!)

User prompt: I filtered the dataset to just larger years, but I still get almost a flat line with that first geom_line, whereas the red line is extremely sloped. what is going on? junH.big<-mu.length %>% filter(Month=="June",Tag=="YES",n>10) dat<-junH.big junH.plot <- ggplot(data = dat, aes(x = Year, y = Lmu)) + ggtitle("Length ~ HW + May/Jun avg") + # --- ALL DATA (Blue Theme) --- geom_line(color = "grey70", alpha = 0.5) + geom_point(aes(size = n), color = "black", alpha = 0.4) + geom_smooth(method = "gam", formula = y ~ s(x, bs = "cs"), aes(weight = n), color = "darkblue", fill = "lightblue", alpha = 0.2) + # --- HIGH SAMPLE DATA ONLY (Red Theme) --- geom_line(data = filter(dat, n > 10), color = "firebrick", linewidth = 0.8) + geom_point(data = filter(dat, n > 10), aes(size = n), color = "firebrick", alpha = 0.9) + geom_smooth(data = filter(dat, n > 10), # <-- Drops low-n years entirely from this model method = "gam", formula = y ~ s(x, bs = "cs"), color = "firebrick", fill = "firebrick", alpha = 0.1) + labs(size = "Sample Size (n)") junH.plot

Response: Ah! Look closely at your background data step.

Because you already preset your core dataset to `dat <- junH.big` (which is *pre-filtered* to only include `n > 10`), **both your gray line and your red line are plotting the exact same data rows.**

So why on earth is the gray line totally flat while the red line is dramatically sloped?

It's a sneaky data-ordering issue. When you ran `filter(Month=="June", Tag=="YES", n>10)` to create `junH.big`, R pulled matching rows but kept them in whatever random order they originally sat in your `mu.length` sheet. They are likely completely out of chronological order.

Why this breaks `geom_line()`:

- **The Gray Line (`geom_line()`):** Connects the points in the *exact sequence they appear in the dataset*. If your years are jumping around (e.g., 2010 -> 1995 -> 2021 -> 2002), R scribbles lines back and forth across the screen. When those dozens of crisscrossing lines overlap with a light opacity (`alpha = 0.5`), they visually mask each other out, making it look like a messy, flat gray blur across the plot area.
- **The Red Line (`data = filter(dat, n > 10)`):** Even though `dat` is out of order, the `filter()` function under the hood sometimes creates a clean, fresh-ordered index or drops rows in a way that shifts the connection order, or your previous script ran a filter step that rearranged them.

The Fix: Sort your data chronologically

To fix this instantly, we just need to make sure `dat` is explicitly sorted by `Year` before it hits `ggplot`. This forces R to draw the lines step-by-step from left to right:

```
junH.big <- mu.length %>%
  filter(Month == "June", Tag == "YES", n > 10) %>%
  arrange(Year) # <-- THIS IS THE CRITICAL LINE

dat <- junH.big

junH.plot <-
  ggplot(data = dat, aes(x = Year, y = Lmu)) +
  ggtitle("June (Tag: YES) - High Sample Size Only") +

  # --- ALL DATA (Gray/Blue Theme - Now nicely ordered) ---
  geom_line(color = "grey70", alpha = 0.5) +
  geom_point(aes(size = n), color = "black", alpha = 0.4) +
  geom_smooth(
    method = "gam", formula = y ~ s(x, bs = "cs"), aes(weight = n),
    color = "darkblue", fill = "lightblue", alpha = 0.2
  ) +

  # --- HIGH SAMPLE DATA ONLY (Red Theme) ---
  geom_line(data = filter(dat, n > 10), color = "firebrick", linewidth = 0.8) +
  geom_point(data = filter(dat, n > 10), aes(size = n), color = "firebrick", alpha = 0.9) +
  geom_smooth(
    data = filter(dat, n > 10),
    method = "gam", formula = y ~ s(x, bs = "cs"),
    color = "firebrick", fill = "firebrick", alpha = 0.1
  ) +

  labs(size = "Sample Size (n)")

junH.plot
```

Once you throw that `arrange(Year)` in there, your gray line will magically wrap itself perfectly underneath your red line!

User prompt: no, they are already sorted chronologically

Response: If the data is already sorted chronologically and the red line is perfectly sloped, then a flat gray line means one thing: **the gray line isn't connecting your years—it is connecting multiple data points *within* each year.**

Check if your `dat` object has multiple rows per year. If it does, `geom_line()` connects all rows in order. If your dataset has multiple records for a single year stacked one after the other, R will draw a jagged, nearly vertical line up and down within each year. When scaled out over decades, these tight vertical lines blend together, completely flattening out the overall sloped trend and looking like a flat gray barcode.

Why the red line looks sloped

When you run `filter(dat, n > 10)`, it drops the years that don't match, but if it also inadvertently reduces your data down to exactly *one row per year*, R can suddenly draw a clean, sloped line from left to right.

The Fix: Force a single path or use a group

If you have multiple observations per year and want a single, clean historical trajectory across your points, you need to explicitly tell `geom_line()` to group by the year or map the structural path cleanly.

To tell `ggplot` to draw the line across the unique years using the mean trajectory, update the gray layer to average things out inline, or apply a grouping aesthetic:

```
junH.plot <-
  ggplot(data = dat, aes(x = Year, y = Lmu)) +
  ggtitle("Length ~ HW + May/Jun avg") +

  # --- FIX: Group or summarize the background line ---
  # If there are duplicate years, this forces R to connect the yearly averages
  stat_summary(fun = mean, geom = "line", color = "grey70", alpha = 0.7, linewidth = 0.8) +

  geom_point(aes(size = n), color = "black", alpha = 0.4) +
  geom_smooth(
    method = "gam", formula = y ~ s(x, bs = "cs"), aes(weight = n),
    color = "darkblue", fill = "lightblue", alpha = 0.2
  ) +

  # --- HIGH SAMPLE DATA ONLY ---
  geom_line(data = filter(dat, n > 10), color = "firebrick", linewidth = 0.8) +
  geom_point(data = filter(dat, n > 10), aes(size = n), color = "firebrick", alpha = 0.9) +
  geom_smooth(
    data = filter(dat, n > 10),
    method = "gam", formula = y ~ s(x, bs = "cs"),
    color = "firebrick", fill = "firebrick", alpha = 0.1
  ) +

  labs(size = "Sample Size (n)")

junH.plot
```

Quick Verification

To confirm this instantly, run this line in your console:

```
any(duplicated(dat$Year))
```

If it returns `TRUE`, you have multiple entries per year causing the flat line visualization issue, and replacing `geom_line()` with `stat_summary(fun = mean, geom = "line")` will fix it.

User prompt: no, just one line per year

Response: If there is strictly **one row per year**, the data is sorted, and you are using the exact same `dat` object for both layers, then a flat gray line alongside a sloped red line is a mathematical impossibility for `geom_line()`.

Because they are reading the exact same data coordinates, they *must* draw the exact same geometric path.

Since they aren't, a silent bug is masking what's actually happening on your screen. Here are the two things causing this visual illusion:

1. The Y-Axis is scaling to zero or blowing out

Look closely at your y-axis limits. When you add `geom_smooth()` or when `dat` was filtered, did your y-axis expand dramatically (e.g., from `0` to `50`)? If your `Lmu` values vary tightly between `14.2` and `14.8`, a sloped line will look completely flat if the graph's y-axis is stretched out wide to accommodate a massive confidence interval from your blue `geom_smooth()` model.

- **Why the red line looks sloped:** The red trend line or its confidence ribbon might be forcing a tight frame on a second plot, or your eyes are tracking the sloped *trend line* rather than the raw point-to-point connections.

2. A Hidden Factor Grouping (The most likely R bug)

If `dat` contains a hidden grouping column (like an active `group_by()` from earlier code, or a variable like `Tag` or `Month` that R thinks has multiple levels), `geom_line()` will secretly split itself into separate lines per group. If it splits your years apart by a factor, it can result in flat, disconnected horizontal segments instead of a single continuous sloped line.

How to diagnose it right now:

Run this exact code block. I have stripped out the complex formatting and explicitly told the gray line to ignore all external groupings using `aes(group = 1)`.

```
# Force group = 1 to break any hidden factor locks
ggplot(data = dat, aes(x = Year, y = Lmu)) +
  geom_line(aes(group = 1), color = "grey70", size = 1) +
  geom_point(aes(size = n), color = "black")
```

If you run just this micro-plot:

- **Does the gray line follow the slope now?** If yes, a hidden grouping variable in `dat` was fracturing your line into flat segments.
- **Is it still flat?** If it's still flat, look at the y-axis numbers. Your points are physically flat relative to the axis scale, and the "slope" you are seeing in the red elements is purely coming from the calculated GAM model line maximizing a tight fit!

User prompt: oh, it is the blue gam line that is flat, not the geom_line

Response: Ah! That makes complete sense. I completely misunderstood which line was giving you trouble.

If the blue GAM line (the one using all the rows in `dat`) is flat, but the red GAM line (using the exact same rows) is sloped, it comes down to one single word in your code: **`aes(weight = n)`**.

In your blue `geom_smooth()` layer, you have:

```
geom_smooth(..., aes(weight = n))
```

Because `dat` only contains years where `n > 10`, you don't have any tiny sample sizes (like $n = 1$ or $n = 2$) left in the dataset. However, you likely have a few years with **massive** sample sizes (e.g., $n = 500$) and other years with modest sample sizes (e.g., $n = 15$).

What the weighting is doing to your model:

When you apply `weight = n`, the GAM algorithm treats a year with $n = 500$ as **33 times more important** than a year with $n = 15$.

If your highest-sample years happen to have very similar `Lmu` values, the weighted GAM will anchor itself entirely to those high-n data points. It will essentially ignore the sloped trajectory of the lower-n years because their statistical weight is next to nothing. The model decides the best-fitting shape is a flat line passing straight through your heavyweight years.

The Red Line Comparison:

In your red `geom_smooth()` layer, you **did not** include `aes(weight = n)`. Because it lacks weights, every single year gets an equal vote ($1:1$) in the shape of the curve. The algorithm tracks the geometric trend of all the points equally, giving you that prominent slope.

The Fix

If you want the blue trend line to match the slope of the red line, you need to remove the weight restriction so all years are evaluated equally:

```
# Change your blue geom_smooth layer to this (remove aes(weight = n)):
geom_smooth(
  method = "gam",
  formula = y ~ s(x, bs = "cs"),
  color = "darkblue",
  fill = "lightblue",
  alpha = 0.2
)
```

Once you remove the weights, both curves will follow the exact same geometric path!

User prompt: why isn't this working? dat<-mu.igf %>%filter(FINAL_GSI=="Interior_Sp") %>% group_by(Year,Month,Tag) %>% summarise(igf=mean(igf),n=n())

Response: The most common reason a `summarise()` block silently fails or gives an error right here is due to **missing values (`NA`)** in your `igf` column.

In R, if a column has even a single `NA` value, functions like `mean()` will automatically return `NA` for that entire group. If your `igf` column contains missing data, your resulting `dat` dataframe will end up with a column full of `NA`s, which can make it look like the calculation didn't work.

The Fix

You need to add `na.rm = TRUE` inside the `mean()` function to tell R to strip out those missing values before calculating the average:

```
dat <- mu.igf %>%
  filter(FINAL_GSI == "Interior_Sp") %>%
  group_by(Year, Month, Tag) %>%
  summarise(
    igf = mean(igf, na.rm = TRUE), # <-- Safely skips any NA values
    n = n()
  )
```

Still not working? Check these two things:

1. **Did you get a "could not find function" error?** If you restarted your R session recently, make sure you run `library(dplyr)` or `library(tidyverse)` at the top of your script so R knows what `group_by` and `summarise` are.
2. **Check your spelling on `FINAL_GSI` values:** R is completely case-sensitive. If the column actually contains `"Interior_sp"` (lowercase "s") or has trailing spaces like `"Interior_Sp "`, your `filter()` step will drop every single row, leaving you with an empty dataset. You can check the exact spelling by running:

   ```
   unique(mu.igf$FINAL_GSI)
   ```

User prompt: why isn't this showing the Month color? ggplot(data = dat, aes(x = Year, y = igf,color=Month)) + ggtitle("IGF ~ HW + May/Jun avg") + # --- ALL DATA (Blue Theme) --- geom_line(color = "grey20", alpha = 0.9) + geom_point(aes(size = n), color = "black", alpha = 0.4) + geom_smooth(method = "gam", formula = y ~ s(x, bs = "cs"), aes(weight = n), color = "darkblue", fill = "lightblue", alpha = 0.2) + # --- HIGH SAMPLE DATA ONLY (Red Theme) --- geom_line(data = filter(dat, n > 10), color = "firebrick", linewidth = 0.8) + geom_point(data = filter(dat, n > 10), aes(size = n), color = "firebrick", alpha = 0.9) + geom_smooth(data = filter(dat, n > 10), # <-- Drops low-n years entirely from this model method = "gam", formula = y ~ s(x, bs = "cs"), color = "firebrick", fill = "firebrick", alpha = 0.1) + facet_wrap(~Tag) + labs(size = "Sample Size (n)")

Response: Your `Month` color isn't showing up because your `geom_line()`, `geom_point()`, and `geom_smooth()` layers are completely overwriting it by declaring explicit, static colors (like `color = "grey20"`, `color = "black"`, and `color = "firebrick"`).

In `ggplot2`, if you specify a solid color string *outside* of `aes()`, it acts as a hard override and completely kills any color mapping you set up in the main `ggplot(aes(color = Month))` call.

If you want your lines and points to be colored by `Month`, you have to move the color mapping into the layers and drop the conflicting hardcoded colors.

Here is the corrected code that preserves your structure but allows the `Month` colors to break through:

```
ggplot(data = dat, aes(x = Year, y = igf, color = Month)) + # color defined here
  ggtitle("IGF ~ HW + May/Jun avg") +

  # --- ALL DATA (Colored by Month automatically) ---
  geom_line(alpha = 0.5, aes(group = Month)) +                # Removed hardcoded gray
  geom_point(aes(size = n), alpha = 0.4) +                    # Removed hardcoded black
  geom_smooth(
    method = "gam", formula = y ~ s(x, bs = "cs"), aes(weight = n, fill = Month),
    alpha = 0.1
  ) +

  # --- HIGH SAMPLE DATA ONLY (Using shapes or linetypes to distinguish from all data) ---
  # Note: If you force these to be "firebrick" red, you will lose the Month color here too!
  geom_line(data = filter(dat, n > 10), aes(group = Month), linewidth = 1.2) +
  geom_point(data = filter(dat, n > 10), aes(size = n), shape = 21, stroke = 1.5) +

  facet_wrap(~Tag) +
  labs(size = "Sample Size (n)", color = "Month", fill = "Month")
```

What changed to fix it:

- **Removed Hardcoded Conflicts:** I stripped `color = "grey20"`, `color = "black"`, and `color = "firebrick"` out of the geoms. Now they safely look back up to `color = Month` in the main header.
- **Added `aes(group = Month)`:** If your data contains both May and June stacked together, `geom_line()` needs to know to draw *two separate lines* (one for May, one for June) per facet, rather than zig-zagging between the months.
- **High-Sample Contrast Trick:** Since we can't use red anymore (or we'd lose the month color), I gave the high-sample layers structural contrast instead. The lines are thicker (`linewidth = 1.2`), and the points use a hollow border style (`shape = 21, stroke = 1.5`) so they pop out visually while keeping their month colors intact!

User prompt: can I just add month to facetwrap?

Response: Yes, absolutely! Faceting by both variables is actually a much cleaner choice here.

By adding `Month` to your `facet_wrap()`, you can split your data into a clean grid of separate panels (e.g., May vs. June across your Tag groups). This eliminates the overlapping data problem entirely, meaning you can keep your original, highly descriptive **blue vs. red** theme to contrast all data against high-sample data.

Here is how you update your code to facet by both variables:

```
ggplot(data = dat, aes(x = Year, y = igf)) +
  ggtitle("IGF ~ HW + May/Jun avg") +

  # --- ALL DATA (Blue Theme) ---
  geom_line(color = "grey20", alpha = 0.5) +
  geom_point(aes(size = n), color = "black", alpha = 0.4) +
  geom_smooth(
    method = "gam", formula = y ~ s(x, bs = "cs"), aes(weight = n),
    color = "darkblue", fill = "lightblue", alpha = 0.2
  ) +
```

```
  # --- HIGH SAMPLE DATA ONLY (Red Theme) ---
  geom_line(data = filter(dat, n > 10), color = "firebrick", linewidth = 0.8) +
  geom_point(data = filter(dat, n > 10), aes(size = n), color = "firebrick", alpha = 0.9) +
  geom_smooth(
    data = filter(dat, n > 10),
    method = "gam", formula = y ~ s(x, bs = "cs"),
    color = "firebrick", fill = "firebrick", alpha = 0.1
  ) +

  # --- MULTI-VARIABLE FACET MATRIX ---
  # This creates a grid layout: Rows = Month, Columns = Tag
  facet_wrap(Month ~ Tag) +

  labs(size = "Sample Size (n)")
```

Why this works beautifully:

- **`facet_wrap(Month ~ Tag)`**: The ~ operator tells R to cross-classify your data. You will get a dedicated plot panel for every combination (e.g., `May : NO`, `May : YES`, `June : NO`, `June : YES`).
- **Preserves Your Model Contrast**: Because the months are now isolated in their own rows or columns, you can clearly see the blue overall-weighted GAM and the red restricted-sample GAM trends side-by-side without the lines clashing or muddying the visualization!

User prompt: I'm back to calculating running means. I have a dataset shark_raw with columns for fmp (spatial area, not to be combined, have to stop averaging if you have entered a new fmp), year, then 4 data columns. I want to make new columns for the avg of year t and t+1 for each data column

Response: To calculate a 2-year forward running mean (averaging year t and year $t+1$) while respecting the boundary of each `fmp` spatial region, you can combine `group_by()` with `across()` and `lead()` from `dplyr`.

Using `lead()` ensures that for any given year, R looks strictly at the next row's value ($t+1$) and calculates the mean with the current year (t). By grouping by `fmp`, R will automatically output `NA` for the final year of a spatial area rather than leaking data across into a new region!

Here is the clean, vectorised solution using `tidyverse`:

```
library(tidyverse)

shark_processed <- shark_raw %>%
  # 1. Ensure data is ordered chronologically within each spatial region
  arrange(fmp, year) %>%

  # 2. Group by fmp so averages never cross spatial boundaries
  group_by(fmp) %>%

  # 3. Calculate (t + t+1) / 2 for all 4 data columns at once
  # Replace 'col1:col4' with your actual 4 column names!
  mutate(across(
    .cols = col1:col4,
    .fns  = ~ (.x + lead(.x, 1)) / 2,
    .names = "{.col}_2yr_avg"
  )) %>%

  # 4. Clean up the grouping metadata
  ungroup()
```

Key details to keep in mind:

1. **`arrange(fmp, year)` is essential:** The `lead()` function relies entirely on row order. Sorting first ensures year $t + 1$ is physically in the row directly below year t.
2. **Boundary Safety:** Because of `group_by(fmp)`, when the code hits the last recorded year for a given `fmp`, `lead()` returns `NA`. This causes the calculation to result in `NA` instead of accidentally averaging with the first year of the next `fmp` area.
3. **Missing Years:** This logic assumes your dataset has continuous, contiguous years (e.g., 2010, 2011, 2012). If your data has missing years in between (e.g., jumps from 2010 to 2013), let me know—we can adjust the code to explicitly check that `lead(year) == year + 1` before averaging!

User prompt: now can you pivot it to wide form with the fmp added to the column name? here is what I just did: shark_processed <- shark_raw %>% select(fmp,year,spiny_dogfish,pacific_sleeper_shark,salmon_shark) %>% # 1. Ensure data is ordered chronologically within each spatial region arrange(fmp, year) %>% # 2. Group by fmp so averages never cross spatial boundaries group_by(fmp) %>% # 3. Calculate (t + t+1) / 2 for all 4 data columns at once # Replace 'col1:col4' with your actual 4 column names! mutate(across(.cols = pacific_sleeper_shark:salmon_shark, .fns = ~ (.x + lead(.x, 1)) / 2, .names = "{.col}_2y_avg_forward")) %>% # 4. Clean up the grouping metadata ungroup()

Response: To pivot your data into wide format where each year gets a single row and the columns reflect the variable, spatial area (`fmp`), and metric type, you can use `pivot_wider()` from `tidyr`.

Here is how you can pipe right from your `shark_processed` dataset into `pivot_wider()`:

```
shark_wide <- shark_processed %>%
  # 1. Pivot the data columns to wide format using fmp in the column names
  pivot_wider(
    id_cols = year,
    names_from = fmp,
    values_from = c(spiny_dogfish, pacific_sleeper_shark, salmon_shark,
                    pacific_sleeper_shark_2y_avg_forward,
                    salmon_shark_2y_avg_forward),
    names_glue = "{fmp}_{.value}"
  ) %>%
  # 2. Keep the table sorted chronologically
  arrange(year)

# View the result
head(shark_wide)
```

What `names_glue` is doing:

- **`{fmp}_{.value}`**: Places the spatial region name at the front of the column name (e.g., `GOA_spiny_dogfish` or `BSAI_pacific_sleeper_shark_2y_avg_forward`).
- If you prefer the variable name first and `fmp` at the end (e.g., `spiny_dogfish_GOA`), you can simply swap the glue pattern to **`"{.value}_{fmp}"`**.

User prompt: what is wrong with this? data_base <- tibble(salmon_dat) %>% left_join(shark_dat %>% select(year, any_of(shark_cands)), by = "year") %>% left_join(sst_dat %>% select(year, any_of(shark_cands)), by = "year")

Response: There is a copy-paste column selection typo in your second `left_join`: you are selecting **`shark_cands`** out of **`sst_dat`** instead of your SST candidate vector (likely named something like `sst_cands`).

Here is what is likely going wrong:

1. **`any_of(shark_cands)` in `sst_dat`:** Because `sst_dat` probably doesn't contain the shark column names, `any_of()` fails silently and returns **only the `year` column**.
2. **Duplicate Column Name Collisions (`.x` / `.y`):** If `sst_dat` *does* share column names with `shark_dat`, joining them both without renaming will cause `left_join` to append `.x` and `.y` to your column names (e.g., `spiny_dogfish.x` and `spiny_dogfish.y`).

The Fix

Replace `shark_cands` with `sst_cands` in the second join:

```
data_base <- tibble(salmon_dat) %>%
  left_join(
    shark_dat %>% select(year, any_of(shark_cands)),
    by = "year"
  ) %>%
  left_join(
    sst_dat %>% select(year, any_of(sst_cands)), # <-- Fixed variable name here
    by = "year"
  )
```

> **Note:** If `salmon_dat` is already a data frame or tibble, wrapping it in `tibble(salmon_dat)` is redundant. You can safely start the pipeline directly with `salmon_dat %>% left_join(...)`.

User prompt: can you also add columns for roll2, where it goes back a year? shark_processed <- shark_raw %>% select(fmp,year,spiny_dogfish,pacific_sleeper_shark,salmon_shark) %>% # 1. Ensure data is ordered chronologically within each spatial region arrange(fmp, year) %>% # 2. Group by fmp so averages never cross spatial boundaries group_by(fmp) %>% # 3. Calculate (t + t+1) / 2 for all 4 data columns at once # Replace 'col1:col4' with your actual 4 column names! mutate(across(.cols = pacific_sleeper_shark:salmon_shark, .fns = ~ (.x + lead(.x, 1)) / 2, .names = "{.col}_2y_avg_forward")) %>% # 4. Clean up the grouping metadata ungroup()

Response: To add a 2-year backward rolling average (averaging year t and year $t-1$), you can use **`lag(.x, 1)`** inside a second `across()` call in your `mutate()` block.

Just like `lead()`, `lag()` will safely return `NA` for the first year within each `fmp` region, ensuring your spatial boundaries are respected.

Here is your updated code:

```
shark_processed <- shark_raw %>%
  select(fmp, year, spiny_dogfish, pacific_sleeper_shark, salmon_shark) %>%
  # 1. Ensure data is ordered chronologically within each spatial region
  arrange(fmp, year) %>%

  # 2. Group by fmp so averages never cross spatial boundaries
  group_by(fmp) %>%

  # 3. Calculate forward (t, t+1) and backward (t, t-1) 2-year averages
  mutate(
    # Forward 2-year average: (Year t + Year t+1) / 2
    across(
      .cols  = spiny_dogfish:salmon_shark,
      .fns   = ~ (.x + lead(.x, 1)) / 2,
      .names = "{.col}_2y_avg_forward"
    ),
    # Backward 2-year average: (Year t + Year t-1) / 2
    across(
      .cols  = spiny_dogfish:salmon_shark,
      .fns   = ~ (.x + lag(.x, 1)) / 2,
```

```
      .names = "{.col}_2y_avg_backward"
    )
  ) %>%

  # 4. Clean up the grouping metadata
  ungroup()
```

Notes on this update:

- **Expanded Column Range:** I updated `.cols = spiny_dogfish:salmon_shark` so that `spiny_dogfish` gets included in both calculations as well (in your previous snippet, it started at `pacific_sleeper_shark`).
- **Boundary Behavior:** For the very first year of an `fmp` area, `_2y_avg_backward` will output `NA`. For the very last year of an `fmp` area, `_2y_avg_forward` will output `NA`.

User prompt: how do I get the x-axis labels to be angled and fit on the page?

Response: To angle your x-axis labels in `ggplot2` so they don't overlap or run off the page, add a `theme()` layer with `axis.text.x = element_text()`.

Here is the standard, best-practice way to do it:

```
your_plot +
  theme(
    axis.text.x = element_text(
      angle = 45,       # Tilt the labels by 45 degrees
      hjust = 1,        # Align the text ends cleanly with the axis ticks
      vjust = 1,        # Ensure proper vertical spacing
      margin = margin(t = 5) # Gives extra buffer room at the bottom
    )
  )
```

Why `hjust = 1` is critical

When you angle text in `ggplot2`, the anchor point defaults to the middle of the text string. Setting **`hjust = 1`** shifts the anchor to the right end of the label, aligning the end of each word/number neatly with its corresponding tick mark instead of floating awkwardly across the axis line.

Common Angles

- **45 Degrees (`angle = 45, hjust = 1`)**: Best overall balance of readability and space-saving.
- **90 Degrees (`angle = 90, hjust = 1, vjust = 0.5`)**: Completely vertical labels. Best if you have extremely dense or very long text labels.

User prompt: shark_choice <- ggplot(shark_imp, aes(shark_col, w, fill = model_group)) + geom_col(position = "dodge") + theme_bw() + labs(title = "AIC-weighted support for overlap form", x = "Overlap form", y = "Summed AIC weight")

Response: Here is your code updated with angled x-axis text using `theme()`.

Since `theme_bw()` resets default styling, make sure to place `theme()` **after** `theme_bw()` so your custom text changes aren't overridden:

```
shark_choice <- ggplot(shark_imp, aes(shark_col, w, fill = model_group)) +
  geom_col(position = "dodge") +
  theme_bw() +
  labs(
```

```
    title = "AIC-weighted support for overlap form",
    x = "Overlap form",
    y = "Summed AIC weight"
  ) +
  theme(
    axis.text.x = element_text(
      angle = 45,       # Rotates the text 45 degrees
      hjust = 1,        # Align text right-edge with axis ticks
      vjust = 1         # Prevents vertical overlap
    ),
    plot.margin = margin(b = 20, r = 10, l = 10, t = 10) # Gives room so labels aren't clipped off
  )

shark_choice
```

Why `plot.margin` helps:

Adding `plot.margin = margin(b = 20, ...)` ensures that if your `shark_col` variable names are very long, `ggplot` adds extra padding at the bottom of the canvas so the tilted text won't be cut off at the edge when rendered or saved with `ggsave()`.

User prompt: this code plots 4 levels of each of 3 overlap forms. can you add the overlap_slope to the y2 axis (the axis along the right hand side of the graph? p_curves <- ggplot() + geom_line(data = curve_tbl, aes(x = T, y = O_T_norm, group = curve_id, color = overlap_form, alpha = aic_weight, linewidth = aic_weight)) + # facet_wrap(~model_group, ncol = 1) + scale_alpha_continuous(range = c(0.08, 0.9), guide = "none") + scale_linewidth_continuous(range = c(0.2, 1.8), guide = "none") + theme_bw() + labs(title = "Shark overlap functional relationship O(T): model family", subtitle = "Each line = one model; alpha/width weighted by within-group AIC weight", x = "Standardized shark temperature driver (T)", y = "Normalized overlap O(T) / mean(O(T))", color = "Overlap form")

Response: To add a secondary y-axis on the right-hand side in `ggplot2`, you can use the **`sec.axis`** argument inside **`scale_y_continuous()`**.

Because secondary axes in `ggplot2` must be a direct mathematical transformation of the primary y-axis, the best approach depends on how `overlap_slope` relates to `O_T_norm`:

Approach 1: Dual-Axis Mapping (Transforming Axis Scale)

If `overlap_slope` is on a completely different numeric scale than `O_T_norm`, you scale the second geom's aesthetic and use `sec_axis(~ . / scale_factor)` to map the right-hand axis correctly:

```
# Define a scale factor based on your data ranges
# (e.g., if max(overlap_slope) is 10 and max(O_T_norm) is 1, scale_factor = 10)
scale_factor <- 1

p_curves <- ggplot() +
  # Primary Curves: Normalized Overlap
  geom_line(
    data = curve_tbl,
    aes(
      x = T, y = O_T_norm, group = curve_id,
      color = overlap_form,
      alpha = aic_weight,
      linewidth = aic_weight
    )
  ) +
  # Secondary Curves: Overlap Slope (scaled to fit primary y-axis)
  geom_line(
    data = curve_tbl,
```

```
    aes(
      x = T, y = overlap_slope / scale_factor, group = curve_id,
      color = overlap_form
    ),
    linetype = "dashed",
    alpha = 0.5
  ) +
  scale_alpha_continuous(range = c(0.08, 0.9), guide = "none") +
  scale_linewidth_continuous(range = c(0.2, 1.8), guide = "none") +
  # Configure both Y axes here
  scale_y_continuous(
    name = "Normalized overlap O(T) / mean(O(T))",
    sec.axis = sec_axis(
      transform = ~ . * scale_factor,
      name = "Overlap slope (dO/dT)"
    )
  ) +
  theme_bw() +
  labs(
    title = "Shark overlap functional relationship O(T): model family",
    subtitle = "Each line = one model; alpha/width weighted by within-group AIC weight",
    x = "Standardized shark temperature driver (T)",
    color = "Overlap form"
  )

p_curves
```

Approach 2: Direct 1:1 Secondary Axis

If `overlap_slope` is already on the exact same numeric scale/range as `O_T_norm` (or if you are plotting a summary value on the right), you don't need a transformation factor:

```
scale_y_continuous(
  name = "Normalized overlap O(T) / mean(O(T))",
  sec.axis = sec_axis(transform = ~ ., name = "Overlap slope")
)
```

Key Tips:

- **Differentiate line styles:** Adding `linetype = "dashed"` to the `overlap_slope` layer makes it immediately obvious to the reader which lines belong to which axis.
- **`transform = ~ . * scale_factor`**: In modern `ggplot2`, use `transform =` inside `sec_axis()` (earlier versions used `trans =`).

User prompt: no, it's not an axis. it is text that shows at the right side of the plot

Response: Ah, got it! You want textual annotations positioned along the right margin of the plot area corresponding to each curve/level, rather than a second numeric y-axis tick scale.

The cleanest way to align text along the right edge of a `ggplot` is using **`geom_text()`** (or `geom_text_repel()` from `ggrepel` if labels overlap). We filter the data to the **maximum x-value** (the rightmost point of each curve, where $T = \max(T)$) so the text anchors cleanly at the far right edge.

Here is how you add those right-hand text annotations:

```
library(tidyverse)
# library(ggrepel) # Optional: uncomment if labels crowd/overlap each other
```

```
# 1. Isolate the rightmost point of each curve to anchor the text labels
right_labels <- curve_tbl %>%
  group_by(curve_id) %>%
  filter(T == max(T)) %>%
  ungroup()

p_curves <- ggplot() +
  # Main curves
  geom_line(
    data = curve_tbl,
    aes(
      x = T, y = O_T_norm, group = curve_id,
      color = overlap_form,
      alpha = aic_weight,
      linewidth = aic_weight
    )
  ) +
  # Right-side text labels
  geom_text(
    data = right_labels,
    aes(
      x = T,
      y = O_T_norm,
      label = round(overlap_slope, 2), # Displays the slope value
      color = overlap_form
    ),
    hjust = -0.2, # Pushes text slightly past the end of the line
    size = 3,
    show.legend = FALSE
  ) +
  scale_alpha_continuous(range = c(0.08, 0.9), guide = "none") +
  scale_linewidth_continuous(range = c(0.2, 1.8), guide = "none") +
  # Allow plot elements to spill into the right margin without getting clipped
  coord_cartesian(clip = "off") +
  theme_bw() +
  theme(
    # Expand right plot margin to give the text room to breathe
    plot.margin = margin(t = 10, r = 40, b = 10, l = 10)
  ) +
  labs(
    title = "Shark overlap functional relationship O(T): model family",
    subtitle = "Each line = one model; alpha/width weighted by within-group AIC weight",
    x = "Standardized shark temperature driver (T)",
    y = "Normalized overlap O(T) / mean(O(T))",
    color = "Overlap form"
  )

p_curves
```

Key parameters making this work:

1. **`hjust = -0.2`**: Offsets the text slightly to the right of the terminal point ($T = \max(T)$) so it doesn't collide directly with the end of the line.
2. **`coord_cartesian(clip = "off")`**: Crucial when placing text at the extreme edge. It prevents `ggplot` from cutting off text that extends outside the panel bounding box.
3. **`plot.margin = margin(..., r = 40)`**: Adds 40 points of blank padding on the right margin so the full text values remain visible.

(Note: If the text values end up overlapping vertically because multiple curves terminate at similar y-values, swap `geom_text(...)` for `ggrepel::geom_text_repel(..., direction = "y", nudge_x = 0.2)` to automatically stack them without overlap!)

User prompt: in this plot, can you rank-order the temperature indices along the x-axis by summed aic weight? sst_imp <- mod_tbl %>% group_by(model_group, shark_temp_col) %>% summarise(w = sum(aic_weight), .groups = "drop") %>% mutate(w_rel = w / max(w)) %>% ungroup() sst_choice <- ggplot(sst_imp, aes(shark_temp_col, w, fill = model_group)) + geom_col(position = "dodge") + theme_bw() + labs(title = "AIC-weighted support for sst index form", x = "sst index", y = "Summed AIC weight") + theme(axis.text.x = element_text(angle = 45, # Rotates the text 45 degrees hjust = 1, # Align text right-edge with axis ticks vjust = 1 # Prevents vertical overlap), plot.margin = margin(b = 20, r = 10, l = 10, t = 10) # Gives room so labels aren't clipped off)

Response: To rank-order categorical x-axis variables in `ggplot2`, convert `shark_temp_col` into a **factor** ordered by total AIC weight using `forcats::fct_reorder()`.

Since your data has multiple `model_group` values per `shark_temp_col`, ordering by total summed AIC weight per index ensures the overall highest-performing indices are sorted first on the x-axis.

Here is the updated code:

```
library(tidyverse)

# 1. Calculate weights and reorder shark_temp_col by total weight
sst_imp <- mod_tbl %>%
  group_by(model_group, shark_temp_col) %>%
  summarise(w = sum(aic_weight), .groups = "drop") %>%
  # Sum weights across model_groups to establish overall rank for ordering
  group_by(shark_temp_col) %>%
  mutate(total_w = sum(w)) %>%
  ungroup() %>%
  # Reorder factor levels descending (.desc = TRUE for highest first)
  mutate(
    shark_temp_col = fct_reorder(shark_temp_col, total_w, .desc = TRUE),
    w_rel = w / max(w)
  )

# 2. Plot with ordered x-axis
sst_choice <- ggplot(sst_imp, aes(x = shark_temp_col, y = w, fill = model_group)) +
  geom_col(position = "dodge") +
  theme_bw() +
  labs(
    title = "AIC-weighted support for sst index form",
    x = "sst index",
    y = "Summed AIC weight"
  ) +
  theme(
    axis.text.x = element_text(
      angle = 45,       # Rotates the text 45 degrees
      hjust = 1,        # Align text right-edge with axis ticks
      vjust = 1         # Prevents vertical overlap
    ),
    plot.margin = margin(b = 20, r = 10, l = 10, t = 10) # Gives room so labels aren't clipped off
  )

sst_choice
```

What changed:

- **`total_w = sum(w)`**: Calculates the combined AIC weight for each temperature index across all model groups.

- **`fct_reorder(shark_temp_col, total_w, .desc = TRUE)`**: Reorders the factor levels of `shark_temp_col` based on `total_w`. Setting `.desc = TRUE` puts the index with the highest overall weight on the far left. If you prefer ascending order (lowest to highest), simply remove `.desc = TRUE`.

---

*User prompt: can you modify this function so that it takes a data_base with year, shark and temperature information, and it adds columns for 2 of the shark _transform options (i.e., the new value after the raw observation has been z-transformed, or "shark_z_roll2"), T_shark, M_t, O_t, and I_shark_z and I_shark_z_roll2? It does not need the log1p_z or the ifelse statements. Here is my start. shark_index_fxn <- function(data_base,col_shark, shark_temp_col, Q10=2, overlap_form, overlap_slope, shark_transform) { # 1. Shark Abundance Transformation if (!col_shark %in% names(data_base)) return(NULL) shark_vec <- data_base[[col_shark]] if (shark_transform == "z") { Shark_use <- safe_scale(shark_vec) } else if (shark_transform == "log1p_z") { minv <- suppressWarnings(min(shark_vec, na.rm = TRUE)) if (!is.finite(minv)) return(NULL) Shark_use <- safe_scale(log1p(shark_vec - minv)) } else if (shark_transform == "z_roll2") { z0 <- safe_scale(shark_vec) zlag <- dplyr::lag(z0, 1) Shark_use <- rowMeans(cbind(z0, zlag), na.rm = TRUE) } else { return(NULL) } # 2. Shark Temperature Driver & Metabolic Response if (!shark_temp_col %in% names(data_base)) return(NULL) T_shark_raw <- data_base[[shark_temp_col]] T_shark <- safe_scale(T_shark_raw) Tref_shark <- mean(T_shark_raw, na.rm = TRUE) M_t <- Q10^((T_shark_raw - Tref_shark) / 10) # 3. Spatial Overlap Form O_t <- if (overlap_form == "constant") { rep(1, nrow(data_base)) } else if (overlap_form == "linear") { pmax(0.1, 1 + overlap_slope * T_shark) } else if (overlap_form == "logistic") { plogis(overlap_slope * T_shark) * 2 } else { return(NULL) } # 4. Construct Integrated Index fit_dat <- data_base %>% mutate(shark_z = safe_scale(shark_vec), shark_z_roll2 = { z0 <- safe_scale(shark_vec) zlag <- dplyr::lag(z0, 1) Shark_use <- rowMeans(cbind(z0, zlag), na.rm = TRUE) }, Tscale=T_shark, Mt = M_t, Ot = O_t, I_Shark_z = safe_scale(shark_vec) * M_t * O_t I_Shark_z_roll2 = Shark_use * M_t * O_t) return(fit_dat) }*

Response: Here is the modified function.

Key Improvements Made:

1. **Removed `shark_transform` parameter and `if/else` logic:** Since we are computing both transformation columns directly inside `mutate()`, we no longer need the transformation selector or initial branching.
2. **Cleaned up `dplyr::mutate()` syntax:** Fixed syntax errors inside the `mutate()` call (missing commas, assigning `Shark_use` inside a curly block) by calculating `shark_z` first and taking advantage of sequential `mutate()` execution to derive `shark_z_roll2`.
3. **Column Validation:** Kept guard clauses at the top to ensure both `col_shark` and `shark_temp_col` exist in `data_base` before computing.

```
shark_index_fxn <- function(data_base,
                            col_shark,
                            shark_temp_col,
                            Q10 = 2,
                            overlap_form,
                            overlap_slope) {

  # 1. Validate Input Columns
  if (!col_shark %in% names(data_base) || !shark_temp_col %in% names(data_base)) {
    return(NULL)
  }

  shark_vec   <- data_base[[col_shark]]
  T_shark_raw <- data_base[[shark_temp_col]]

  # 2. Temperature Driver & Metabolic Response
  T_shark    <- safe_scale(T_shark_raw)
  Tref_shark <- mean(T_shark_raw, na.rm = TRUE)
  M_t        <- Q10^((T_shark_raw - Tref_shark) / 10)

  # 3. Spatial Overlap Form
  O_t <- if (overlap_form == "constant") {
```

```
    rep(1, nrow(data_base))
  } else if (overlap_form == "linear") {
    pmax(0.1, 1 + overlap_slope * T_shark)
  } else if (overlap_form == "logistic") {
    plogis(overlap_slope * T_shark) * 2
  } else {
    return(NULL)
  }

  # 4. Construct Integrated Dataframe
  fit_dat <- data_base %>%
    mutate(
      # Transformations
      shark_z       = safe_scale(shark_vec),
      shark_z_roll2 = rowMeans(cbind(shark_z, dplyr::lag(shark_z, 1)), na.rm = TRUE),

      # Temperature and Ecological Drivers
      T_shark       = T_shark,
      M_t           = M_t,
      O_t           = O_t,

      # Integrated Indices
      I_shark_z       = shark_z * M_t * O_t,
      I_shark_z_roll2 = shark_z_roll2 * M_t * O_t
    )

  return(fit_dat)
}
```

Why sequential `mutate()` works cleanly here:

In `dplyr::mutate()`, newly defined variables are immediately available to subsequent arguments within the same function call. Defining `shark_z = safe_scale(shark_vec)` first allows us to reuse `shark_z` directly inside `lag()`, `I_shark_z`, and `I_shark_z_roll2` without repetitive code execution.

User prompt: what would it do to the overlap function if the slope were 1?

Response: It depends on which **`overlap_form`** ("linear" or "logistic") you pass into your function! Here is what setting `overlap_slope = 1` mathematically does to both options:

1. Linear Form (`overlap_form = "linear"`)

Your code calculates:

$$\text{O\_t} = \max(0.1, 1 + 1 \times T_{shark})$$

Because T_{shark} is z-score standardized ($\text{mean} = 0, \text{SD} = 1$):

- **At average temperatures ($T_{shark} = 0$):**

  $\text{O\_t} = 1 + (1 \times 0) = \mathbf{1.0}$ (baseline overlap).

- **Warm years ($T_{shark} = +1$ SD):**

  $\text{O\_t} = 1 + (1 \times 1) = \mathbf{2.0}$ (**doubles** baseline overlap).

- **Cold years ($T_{shark} = -1$ SD):**

$\text{O\_t} = 1 + (1 \times -1) = 0$, which gets capped by `pmax(0.1, ...)` at 0.1.

In short: A slope of 1 creates a **steep positive response**. For every 1 SD increase in temperature, spatial overlap increases by 100% of its baseline value, bottoming out at the 0.1 minimum floor once temperatures drop below average.

2. Logistic Form (`overlap_form = "logistic"`)

Your code calculates:

$$\text{O\_t} = \text{plogis}(1 \times T_{shark}) \times 2 = \frac{2}{1 + e^{-T_{shark}}}$$

- **At average temperatures ($T_{shark} = 0$):**

  $\text{O\_t} = 0.5 \times 2 = \mathbf{1.0}$ (baseline overlap).

- **Warm years ($T_{shark} = +1$ SD):**

  $\text{O\_t} \approx 0.731 \times 2 = \mathbf{1.46}$ (46% increase).

- **Cold years ($T_{shark} = -1$ SD):**

  $\text{O\_t} \approx 0.269 \times 2 = \mathbf{0.54}$ (46% decrease).

  In short: A slope of 1 creates a **moderately strong S-shaped curve**. Overlap varies smoothly between an absolute minimum of 0 (extreme cold) and a maximum ceiling of 2.0 (extreme heat), with the fastest rate of change happening near average temperatures.

Comparison Summary

Metric / Scenario	Linear (`slope = 1`)	Logistic (`slope = 1`)
Average Temp ($T = 0$)	1.0	1.0
Warm Year ($T = +1$ SD)	2.0	1.46
Cold Year ($T = -1$ SD)	0.1 (capped floor)	0.54
Upper Limit	Unbounded ($\propto T$)	Bounded at 2.0
Lower Limit	Hard floor at 0.1	Asymptote at 0.0

User prompt: when I plot M_t and O_t, they are almost identical and very close to 1. so it seems like what I am actually doing in the function is squaring it. that doesn't seem to be adding much information. > newdat year CPUE_Juv SAR_Adult sar_sr sar_uc Hake goa_pacific_sleeper_shark enso_dj pdo_djf sst_wgoa_coastwatch_junjulaug 1 1998 -0.42324500 0.55256724 -3.955270 NA -0.31671257 74 2.20 1.02000000 10.417609 2 1999 1.76786304 1.05874919 -3.251305 NA -0.92618321 558 -1.30 -0.47333333 9.655000 3 2000 0.19906981 0.57756768 -3.417493 NA -1.06163350 608 -1.30 -1.48666667 10.371630 4 2001 -0.40212329 -1.55029462 -6.199218 NA -0.78062227 249 -0.80 0.47000000 11.132391 5 2002 -0.02145001 -0.13653021 -4.006052 0.5476048753 -0.21121715 226 0.10 -0.43333333 10.549783 6 2003 0.05152708 -1.04702937 -4.722712 -0.2079991485 0.66801935 270 0.80 1.98000000 11.042609 7 2004 -0.39280862 -1.42537142 -4.999181 -0.3465404295 1.96012166 282 0.20 0.41333333 11.531413 8 2005 -0.41363740 -1.43330972 -5.620958 NA 1.26060966 482 0.10 0.36000000 12.075870 9 2006 -0.24247540 0.13114863 -3.699228 NA 0.26082054 252 -0.70 0.63000000 10.290652 10 2007 -0.18947734 -0.05773978 -3.888075 -0.0007689351 -1.56020325 295 0.60 0.06333333 10.160435 11 2008 2.55243519 1.38330923 -2.679386 0.9747869386 -2.56798497 66 -1.10

-0.78333333 9.611739 12 2009 -0.05842685 1.22658417 -3.414066 1.0248128057 -1.53839204 56 -1.00 -1.27333333 9.884239 13 2010 -0.01840906 1.08637986 -3.801621 1.2362336665 -1.45850396 170 0.90 0.57666667 9.728913 14 2011 -0.18628378 0.13802119 -4.441503 0.4796500063 -0.81255006 26 -1.80 -0.98666667 9.960978 15 2012 0.32846097 0.71537469 -3.734555 0.8558725297 -1.17170060 180 -1.10 -1.34000000 9.509130 16 2013 0.53121706 0.74901710 -3.554662 0.9034425894 -0.05838167 94 -0.43 -0.34666667 10.428913 17 2014 -0.31527001 -0.61560776 -4.677778 0.0437197074 0.11354854 93 -0.42 0.09000000 11.405652 18 2015 -0.25010925 -0.86887283 -5.222092 0.2092865167 0.89405731 60 0.55 2.42000000 11.565000 19 2016 -0.36061674 -0.55815344 -4.439659 0.1577176399 1.02954175 71 2.48 1.43000000 12.333370 20 2017 -0.52042375 -0.55795754 -4.881286 0.3853283121 1.07601233 140 -0.34 0.88000000 10.993370 21 2018 -0.38062831 -0.75882481 -4.693768 0.0062097166 1.11246487 250 -0.92 0.52333333 10.474348 22 2019 -0.37386312 -0.36822038 -4.560095 0.2136648074 1.38277800 92 0.75 0.54666667 12.101413 23 2020 -0.41910582 0.91744297 -3.639035 1.1358894372 1.08674025 98 0.50 0.02000000 11.519891 24 2021 -0.46138096 0.46755099 -4.087136 0.7067605502 1.13605835 91 -1.05 -0.53333333 10.428571 sst_egoa_coastwatch_junjulaug shark_z shark_z_roll2 T_shark M_t O_t l_shark_z l_shark_z_roll2 1 12.15576 -0.7851499 -0.78514991 2.1419870 1.1751406 1.6946026 -1.56354459 -1.56354459 2 11.56033 2.2478735 0.73136179 -1.0778933 0.9219966 0.5937120 1.23048688 0.40034775 3 12.40783 2.5612024 2.40453792 -1.0778933 0.9219966 0.5937120 1.40200323 1.31624506 4 12.23924 0.3115011 1.43635173 -0.6179104 0.9545107 0.7577489 0.22530237 1.03888375 5 11.79902 0.1673698 0.23943547 0.2100589 1.0159528 1.0838264 0.18429367 0.26364633 6 12.76924 0.4430992 0.30523453 0.8540349 1.0664626 1.3289175 0.62797831 0.43259083 7 13.94511 0.5182982 0.48069870 0.3020554 1.0230193 1.1202377 0.59398251 0.55089260 8 13.77826 1.7716136 1.14495589 0.2100589 1.0159528 1.0838264 1.95075280 1.26072971 9 12.41554 0.3303008 1.05095723 -0.5259138 0.9611499 0.7926837 0.25165219 0.80071151 10 12.30750 0.5997637 0.46503226 0.6700418 1.0517803 1.2617784 0.79595454 0.61715065 11 10.87772 -0.8352825 -0.11775943 -0.8939001 0.9348672 0.6569370 -0.51298777 -0.07232181 12 12.05848 -0.8979483 -0.86661542 -0.8019035 0.9413697 0.6898048 -0.58309289 -0.56274653 13 11.73652 -0.1835585 -0.54075340 0.9460315 1.0738804 1.3613281 -0.26834482 -0.79052933 14 12.36609 -1.0859456 -0.63475206 -1.5378762 0.8905900 0.4522575 -0.43739286 -0.25566291 15 11.39935 -0.1208927 -0.60341917 -0.8939001 0.9348672 0.6569370 -0.07424612 -0.37058916 16 12.93663 -0.6598184 -0.39035554 -0.2775230 0.9793072 0.8894445 -0.57472780 -0.34001506 17 12.74120 -0.6660849 -0.66295165 -0.2683234 0.9799862 0.8930809 -0.58296217 -0.58021990 18 13.30663 -0.8728820 -0.76948347 0.6240435 1.0481414 1.2445590 -1.13865165 -1.00377099 19 13.68304 -0.8039496 -0.83841582 2.3995775 1.1981706 1.7442015 -1.68013490 -1.75216406 20 12.63620 -0.3715558 -0.58775273 -0.1947261 0.9854355 0.9222667 -0.33768268 -0.53416986 21 12.56522 0.3177677 -0.02689406 -0.7283063 0.9466043 0.7166484 0.21556802 -0.01824446 22 13.45880 -0.6723515 -0.17729192 0.8080366 1.0627729 1.3124107 -0.93779223 -0.24728580 23 12.56120 -0.6347521 -0.65355179 0.5780452 1.0445151 1.2271839 -0.81363285 -0.83773057 24 12.58143 -0.6786181 -0.65668508 -0.8479018 0.9381128 0.6732721 -0.42861870 -0.41476569> dat=newdat[, c("year","enso_dj", "pdo_djf","M_t","O_t")]> n=ncol(dat)-1> matplot(dat[,1], (dat[,-1]),type='b',col=1:n,lty=1:n)> legend("top",legend=paste(1:n,names(dat[,-1])),col=1:n,bty='n',cex=0.7)

Response: You hit on a fundamental issue in ecological index construction: **when two driver components are linear functions of the same standardized variable, multiplying them together doesn't give you two independent mechanisms—it just creates a quadratic function of temperature.**

1. Why `M_t` and `O_t` Look Identical

Look at the mathematical formulas inside your function:

1. **Metabolic Response (M_t):**

$$M_t = Q_{10}^{\frac{T_{raw} - \bar{T}}{10}}$$

   Because real-world sea surface temperatures typically vary by only ± 1 to $2\,^{\circ}\mathrm{C}$ around the mean, the exponent $\frac{T_{raw} - \bar{T}}{10}$ stays tiny (between -0.2 and $+0.2$). The Taylor expansion of Q_{10}^{x} for small x is **almost perfectly linear**.

2. **Linear Overlap (O_t):**

$$O_t = 1 + \text{slope} \times T_{shark}$$

   This is explicitly a **linear function** of temperature.

When you multiply two nearly linear functions of the same temperature driver T:

$$\text{Temperature Multiplier} = M_t \times O_t \approx (1 + aT)(1 + bT) = 1 + (a + b)T + abT^2$$

You aren't capturing two distinct ecological mechanisms; you are simply taking temperature T, scaling it, and adding a slight T^2 curvature!

2. Three Ways to Make the Index Ecologically Meaningful

Option A: Use Different Spatial/Temporal Drivers

In marine ecology, metabolic rate (M_t) is driven by **local temperature** where the shark lives, but spatial overlap (O_t) is usually driven by **thermal habitat volume, spatial SST gradients, or regional climate indices** (like PDO/ENSO or coastal thermal fronts).

If O_t is calculated from a spatial index or a different environmental column rather than the exact same T_{shark} variable, $M_t \times O_t$ will capture genuine, independent variation.

Option B: Make O_t Non-Linear (Threshold or Optimum)

Sharks don't usually experience infinite linear increases in overlap with temperature—they have thermal preferences. A **Gaussian/Optimum** or **S-shaped Logistic** overlap curve prevents O_t from mirroring M_t:

```
# Non-linear Overlap: Thermal Optimum Model
# Overlap peaks when T is near an optimal temperature (e.g., T_scale = 0)
O_t_optimum <- exp(-0.5 * (T_shark / overlap_slope)^2)
```

Option C: Drop O_t or Simplify to a Single Temperature Multiplier

If O_t cannot be parameterized with independent spatial data, having both M_t and O_t in the same equation creates collinearity and over-parameterizes your model.

You can collapse the temperature effect into a single **Temperature Modulated Activity Index (I_{shark})**:

$$I_{shark} = \text{Shark Abundance} \times M_t$$

Where M_t represents the combined metabolic/encounter-rate thermal driver.

Revised Function Code

Here is how you can update your function to give O_t a distinct non-linear shape (Optimum/Gaussian) or allow an independent spatial driver column (`overlap_temp_col`):

```
shark_index_fxn <- function(data_base,
                            col_shark,
                            shark_temp_col,
                            overlap_temp_col = NULL, # Optional separate driver for overlap
                            Q10 = 2,
                            overlap_form = "linear",
                            overlap_slope = 1) {

  if (!col_shark %in% names(data_base) || !shark_temp_col %in% names(data_base)) {
    return(NULL)
  }

  shark_vec   <- data_base[[col_shark]]
  T_shark_raw <- data_base[[shark_temp_col]]

  # 1. Metabolic Response (Temperature driver)
  T_shark    <- safe_scale(T_shark_raw)
  Tref_shark <- mean(T_shark_raw, na.rm = TRUE)
```

```
  M_t        <- Q10^((T_shark_raw - Tref_shark) / 10)

  # 2. Overlap Driver (Uses overlap_temp_col if provided, otherwise T_shark)
  T_overlap_raw <- if (!is.null(overlap_temp_col) && overlap_temp_col %in% names(data_base)) {
    data_base[[overlap_temp_col]]
  } else {
    T_shark_raw
  }
  T_overlap <- safe_scale(T_overlap_raw)

  # 3. Spatial Overlap Form
  O_t <- if (overlap_form == "constant") {
    rep(1, nrow(data_base))
  } else if (overlap_form == "linear") {
    pmax(0.1, 1 + overlap_slope * T_overlap)
  } else if (overlap_form == "logistic") {
    plogis(overlap_slope * T_overlap) * 2
  } else if (overlap_form == "optimum") {
    # Gaussian thermal optimum centered at mean temp
    exp(-0.5 * (T_overlap / overlap_slope)^2)
  } else {
    return(NULL)
  }

  # 4. Construct Integrated Dataframe
  fit_dat <- data_base %>%
    mutate(
      shark_z       = safe_scale(shark_vec),
      shark_z_roll2 = rowMeans(cbind(shark_z, dplyr::lag(shark_z, 1)), na.rm = TRUE),

      T_shark       = T_shark,
      M_t           = M_t,
      O_t           = O_t,

      I_shark_z       = shark_z * M_t * O_t,
      I_shark_z_roll2 = shark_z_roll2 * M_t * O_t
    )

  return(fit_dat)
}
```
